

Python Programming Foundations

5-Day Beginner Python Course

CANDIDATE EDITION

Table of Contents

1. [Course Overview](#)
2. [Day 1: Python Basics - Data Types, Variables & Operators](#)
3. [Day 2: Expressions, Statements & Conditions in Python](#)
4. [Day 3: Loops, Lists & Data Structures in Python](#)
5. [Day 4: Functions, Scope & Errors in Python](#)
6. [Day 5: Input/Output, Algorithms & Complete Python Program](#)
7. [Final Assessment](#)
8. [Python Quick Reference](#)
9. [Glossary](#)

Course Overview

What You Will Learn

By the end of this 5-day Python course, you will be able to:

- Write and run Python code
- Work with Python's data types (int, float, str, bool, list, dict, None)
- Create and use variables (immutable mindset)
- Write expressions using Python operators
- Use conditions (if/elif/else) for decision-making
- Write loops (for, while) for repetition
- Work with lists, tuples, and dictionaries
- Create and use functions
- Understand scope in Python
- Handle errors with try/except
- Handle input and output

- Write complete Python programs

Course Materials

You will need:

- Python installed (version 3.6+)
- A code editor (VS Code, PyCharm, or IDLE)
- Notebook for notes
- Internet connection (for practice)

How to Set Up Python

1. **Download Python:** <https://www.python.org/downloads/>
2. **Install:** Follow installation instructions for your OS
3. **Verify:** Open terminal/command prompt and type `python --version`
4. **Editor:** Install VS Code (free) or use IDLE (comes with Python)

Day 1: Python Basics

Data Types, Variables & Operators

Learning Objectives

By the end of Day 1, you will be able to:

- Identify Python's 5 core data types
- Create and use variables (immutable mindset)
- Write Python expressions using operators
- Spot and fix type errors in Python

1.1 Python Data Types

What Are Data Types in Python?

Python has several built-in data types. Everything in Python is an object with a type.

The 5 Core Data Types

1. Integer (int)

Whole numbers, positive or negative.

Examples:

```
5
-10
1000
0
```

Operations:

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: / (returns float)
- Floor Division: // (returns int)
- Modulus (remainder): %
- Exponentiation: **

2. Float (float)

Numbers with decimal points.

Examples:

```
3.14
-0.5
100.0
2.71828
```

Operations: Same as integers, but always returns a float.

3. String (str)

Text data, enclosed in quotes.

Examples:

```
"Hello"
'John'
"The sky is blue"
```

```
"123" # This is a string, not a number!  
'True' # This is a string, not a boolean!
```

Operations:

- Concatenation: +
- Repetition: *
- Length: len()
- Indexing: [0]
- Slicing: [0:3]
- Methods: .upper(), .lower(), .strip(), .replace()

4. Boolean (bool)

True or False.

Examples:

```
True  
False
```

Operations:

- and (both must be True)
- or (at least one True)
- not (opposite)

5. List (list)

Ordered, mutable collection.

Examples:

```
[1, 2, 3, 4, 5]  
["apple", "banana", "cherry"]  
[True, False, True]  
[10, "hello", True] # Mixed types (allowed but not recommended)  
[] # Empty list
```

Operations:

- Length: len()

- Indexing: `[0]`
- Slicing: `[0:3]`
- Append: `.append()`
- Extend: `.extend()`
- Insert: `.insert()`
- Remove: `.remove()`
- Pop: `.pop()`

6. Dictionary (dict)

Key-value pairs.

Examples:

```
{"name": "Alice", "age": 25}
{"apple": 1.00, "banana": 2.00}
{} # Empty dictionary
```

Operations:

- Access: `["key"]`
- Get: `.get()`
- Keys: `.keys()`
- Values: `.values()`
- Items: `.items()`

7. NoneType (None)

Represents "nothing" or "no value".

Examples:

`None`

Quick Reference: Python Data Types

Type	Python	Example	Operations
Integer	<code>int</code>	25	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>//</code> , <code>%</code> , <code>**</code>

Type	Python	Example	Operations
Float	float	3.14	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code>
String	str	"Hello"	<code>+</code> , <code>*</code> , <code>len()</code> , <code>.upper()</code>
Boolean	bool	True	<code>and</code> , <code>or</code> , <code>not</code>
List	list	[1,2,3]	<code>len()</code> , <code>[0]</code> , <code>.append()</code>
Dictionary	dict	{"a":1}	<code>["key"]</code> , <code>.get()</code>
None	NoneType	None	Check with <code>is None</code>

1.2 Variables in Python

What Is a Variable?

A **variable** is a name that points to a value in memory.

Syntax:

```
variable_name = value
```

Examples:

```
age = 25
name = "John"
is_student = True
scores = [85, 90, 78]
person = {"name": "Alice", "age": 30}
```

Variable Naming Rules

1. Must start with a letter or underscore (`_`)
2. Can contain letters, numbers, underscores
3. Case-sensitive (`age` $\neq$ `Age`)
4. Cannot use Python keywords (`if` , `for` , `while` , etc.)

Good variable names:

```
user_age
first_name
total_score
```

```
is_active
item_list
```

Bad variable names:

```
2name      # Can't start with number
user-name  # No hyphens
if         # Reserved keyword
class      # Reserved keyword
```

The Immutable Mindset

In functional programming, **variables never change**. Python allows mutation, but we'll practice the FP approach.

WRONG (mutation):

```
age = 25
age = 26 # Bad practice in FP
```

RIGHT (immutable):

```
age = 25
new_age = 26 # Create a new variable
```

Type Checking

You can check the type of a variable with `type()` :

```
age = 25
print(type(age)) # <class 'int'>

name = "John"
print(type(name)) # <class 'str'>
```

Type Conversion

Convert between types:

```
# String to int
int("123") # 123

# Int to string
str(456) # "456"
```

```
# Float to int
int(3.14) # 3 (truncates)

# String to float
float("3.14") # 3.14

# List to string
str([1, 2, 3]) # "[1, 2, 3]"
```

1.3 Python Operators

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	3 * 4	12
/	Division (float)	20 / 5	4.0
//	Floor Division (int)	17 // 5	3
%	Modulus (remainder)	17 % 5	2
**	Exponentiation	2 ** 3	8

Examples:

```
# Arithmetic in action
print(5 + 3) # 8
print(10 - 4) # 6
print(3 * 4) # 12
print(20 / 5) # 4.0
print(17 // 5) # 3
print(17 % 5) # 2
print(2 ** 3) # 8
```

Comparison Operators

Operator	Name	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	10 > 5	True
<	Less than	3 < 8	True
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	4 <= 3	False

Examples:

```

print(10 > 5)      # True
print(3 < 8)       # True
print(5 == 5)      # True
print(5 == "5")    # False (different types)
print(5 != 3)      # True
print(5 >= 5)      # True

```

Logical Operators

Operator	Name	Example	Result
and	AND	True and True	True
and	AND	True and False	False
or	OR	True or False	True
or	OR	False or False	False
not	NOT	not True	False
not	NOT	not False	True

Examples:

```

# AND – both must be True
print(True and True)    # True
print(True and False)   # False

# OR – at least one must be True
print(True or False)    # True
print(False or False)   # False

```

```
# NOT – reverses
print(not True)      # False
print(not False)     # True

# Combining comparisons
age = 25
print(age >= 18 and age < 65)  # True (adult working age)
```

String Operators

Operator	Name	Example	Result
+	Concatenation	"Hello" + " World"	"Hello World"
*	Repetition	"Hi" * 3	"HiHiHi"
in	Membership	"a" in "apple"	True
[]	Indexing	"Hello"[0]	"H"
[:]	Slicing	"Hello"[1:3]	"el"

Examples:

```
name = "John"
greeting = "Hello"
print(greeting + " " + name)  # "Hello John"
print("Ha" * 5)               # "HaHaHaHaHa"
print("a" in "apple")         # True
print("Hello"[0])             # "H"
print("Hello"[1:4])           # "ell"
```

List Operators

Operator	Name	Example	Result
+	Concatenation	[1,2] + [3,4]	[1,2,3,4]
*	Repetition	[0] * 5	[0,0,0,0,0]
in	Membership	2 in [1,2,3]	True
[]	Indexing	[1,2,3][0]	1
[:]	Slicing	[1,2,3][0:2]	[1,2]

Examples:

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
print(list1 + list2)      # [1, 2, 3, 4, 5, 6]
print([0] * 5)            # [0, 0, 0, 0, 0]
print(2 in [1, 2, 3])    # True
print([1, 2, 3][1])      # 2
print([1, 2, 3, 4][1:3]) # [2, 3]
```

1.4 Type Errors in Python

What Is a TypeError?

A **TypeError** occurs when you try to perform an operation on the wrong type of data.

Common TypeErrors

Error 1: String × Integer

```
"Hello" * 3    # Works! Repeats string
"Hello" + 3    # TypeError! Can't add string and int
```

Error 2: Number + String

```
5 + 3          # Works (8)
5 + "3"        # TypeError! Can't add int and str
```

Error 3: List + String

```
[1,2] + [3,4]   # Works ([1,2,3,4])
[1,2] + "hello" # TypeError! Can't add list and str
```

Error 4: Division by Zero

```
10 / 0          # ZeroDivisionError!
```

How to Avoid TypeErrors

1. **Check your types:** Use `type()` to debug
2. **Convert explicitly:** Use `int()`, `str()`, `float()` as needed

3. Don't mix types: Unless you're sure

Type Coercion in Python

Python automatically converts some types:

```
# int + float → float
5 + 3.14      # 8.14 (float)

# int / int → float
5 / 2         # 2.5 (float)

# int // int → int (floor division)
5 // 2        # 2 (int)
```

Note: Python does NOT automatically convert int to str or vice versa:

```
5 + "3"       # TypeError!
str(5) + "3"   # "53" (works)
5 + int("3")   # 8 (works)
```

Day 1 Exercises

Exercise 1.1: Identify Python Data Types

Instructions: What is the data type of each value?

Value	Python Type
25	
"Hello"	
True	
[1, 2, 3]	
None	
3.14	
{"name": "Alice"}	
"123"	

Exercise 1.2: Create Variables

Instructions: Create Python variables for the following values.

Value	Variable Name	Python Code
Number 25	age	
Text "John"	first_name	
Boolean True	is_student	
List [1,2,3]	scores	
Dictionary {"name":"Alice"}	person	

Change the value (immutable style):

Original	Change to	New Variable
age = 25	26	
name = "John"	"Jane"	
is_active = True	False	

Exercise 1.3: Python Calculator

Instructions: Write Python expressions to solve these problems. Write the result.

Expression	Python Code	Result
$5 + 3 \times 2$		
$10 \div 3$ (float)		
$10 \div 3$ (integer)		
2 to the power of 4		
Remainder of $17 \div 5$		
"Hello" + " " + "World"		
"Hi" repeated 3 times		
$[1,2] + [3,4]$		

Exercise 1.4: Type Checking

Instructions: Write Python code to check the type of each variable.

```
x = 5
y = "Hello"
z = True
w = [1, 2, 3]
v = {"name": "Alice"}
u = None
```

Your code:

```
print(type(x)) #
print(type(y)) #
print(type(z)) #
print(type(w)) #
print(type(v)) #
print(type(u)) #
```

Exercise 1.5: Fix Type Errors

Instructions: Each expression has an error. Fix it.

```
# 1. TypeError
result = 5 + "3"
```

```
# Fixed:
```

```
# 2. TypeError
result = [1, 2] + 3
```

```
# Fixed:
```

```
# 3. ZeroDivisionError
result = 10 / 0
```

```
# Fixed:
```

```
# 4. TypeError
result = "Hello" + 5
```

```
# Fixed:
```

Exercise 1.6: Complete Python Data Map

Instructions: Design a Python data map for a student management system.

Scenario:

- Students have names, ages, and grades
- Grades are a list of numbers
- Each student has a passing status
- Sometimes grades are missing

Data Type	Variables (Python)	Operations
int		
float		
str		
bool		
list		
dict		
None		

Day 1 Summary

Key Takeaways

1. **Python Data Types:** `int` , `float` , `str` , `bool` , `list` , `dict` , `None`
2. **Variables:** Names that point to values
3. **Immutable Mindset:** Create new variables instead of changing existing ones
4. **Operators:** Arithmetic (`+` , `-` , `*` , `/`), Comparison (`>` , `<` , `==`), Logical (`and` , `or` , `not`)
5. **Type Errors:** Python raises `TypeError` when you mix incompatible types

Quiz Yourself

1. What are the 7 core Python data types?
2. What is the difference between `/` and `//` ?
3. What is the difference between `==` and `=` ?
4. How do you check the type of a variable?
5. What is a `TypeError` ?

Day 2: Python Expressions, Statements & Conditions

Learning Objectives

By the end of Day 2, you will be able to:

- Distinguish between Python expressions and statements
- Write Python expressions that produce values
- Use `if`, `elif`, and `else` for decision-making
- Build Python programs with conditions

2.1 Python Expressions

What Is an Expression in Python?

An **expression** is any combination of values, variables, and operators that produces a value.

Examples:

```
5 + 3                # Produces 8
"Hello" + " World"   # Produces "Hello World"
age >= 18             # Produces True or False
len(name)            # Produces the length
scores[0]            # Produces the first score
```

Characteristics of Expressions in Python

- Always produce a value
- Can be used wherever a value is expected
- Can be nested (expressions inside expressions)

Examples of Python Expressions

Expression	Produces
5 + 3	8
"Hello" + "!"	"Hello!"

Expression	Produces
age >= 18	True or False
scores[0]	First score
len(name)	Number of characters
2 ** 3	8
a + b * c	Depends on values

Expression vs. Statement in Python

Expression	Statement
Produces a value	Performs an action
Can be used in expressions	Standalone command
Example: 5 + 3	Example: print("Hello")
Example: age >= 18	Example: x = 5 (assignment)

Note: In Python, assignment (=) is a statement, not an expression.

2.2 Python Statements

What Is a Statement in Python?

A **statement** is a command that Python executes. It doesn't produce a value (or produces `None`).

Types of Statements

1. Assignment Statement:

```
x = 5
name = "John"
```

2. Print Statement (Function):

```
print("Hello")
print(x + 3)
```

3. Condition Statements:

```
if x > 10:
    print("x is big")
```

4. Loop Statements:

```
for item in list:
    print(item)
```

5. Function Definition Statement:

```
def greet(name):
    return "Hello " + name
```

Functional Programming Rule for Python

Use expressions for ALL logic. Use statements ONLY for input/output and control flow.

Good Python (FP Style):

```
# All calculations are expressions
subtotal = sum(prices)           # Expression
discount = subtotal * 0.1        # Expression
final_price = subtotal - discount # Expression
print(final_price)               # Statement (only at the end)
```

Bad Python (too many statements):

```
# Many mutation statements
total = 0           # Statement
for price in prices: # Statement
    total = total + price # Statement
print(total)        # Statement
```

2.3 Python Conditions

if Statement

The `if` statement allows you to make decisions.

Syntax:

```
if condition:
    # code to run if condition is True
```

Example:

```
age = 25
if age >= 18:
    print("Adult")
```

if-else Statement

Syntax:

```
if condition:
    # code if True
else:
    # code if False
```

Example:

```
age = 16
if age >= 18:
    print("Adult")
else:
    print("Child")
```

if-elif-else Statement

Syntax:

```
if condition1:
    # code if condition1 True
elif condition2:
    # code if condition2 True
else:
    # code if all conditions False
```

Example:

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
```

```
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade) # "B"
```

Chaining Conditions with and , or , not

```
age = 25
is_student = True
is_employed = False

# AND – both must be True
if age >= 18 and age < 65:
    print("Working age adult")

# OR – at least one must be True
if is_student or is_employed:
    print("Active person")

# NOT – reverses
if not is_employed:
    print("Not employed")
```

Nested Conditions

```
age = 25
is_student = True
has_id = True

if age >= 18:
    if is_student and has_id:
        print("Student discount")
    else:
        print("Regular price")
else:
    print("Child price")
```

2.4 Python Condition Examples

Example 1: Voting Eligibility

```
age = 25
```

```
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

Example 2: Grade Calculator

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Score: {score}, Grade: {grade}")
```

Example 3: Discount Eligibility

```
total = 120
is_member = True

if is_member:
    if total > 100:
        discount = 0.2 # 20% for members with high spend
    else:
        discount = 0.1 # 10% for members
else:
    if total > 100:
        discount = 0.05 # 5% for non-members with high spend
    else:
        discount = 0 # no discount

final_price = total * (1 - discount)
print(f"Original: ${total}, Discount: {discount*100}%, Final: ${final_price}")
```

Example 4: Weather Decision Tree

```
temperature = 25

if temperature > 30:
    print("Hot")
```

```
elif temperature > 20:  
    print("Warm")  
elif temperature > 10:  
    print("Cool")  
else:  
    print("Cold")
```

Day 2 Exercises

Exercise 2.1: Expression or Statement?

Instructions: Identify each as expression (E) or statement (S).

Code	E or S?	Why?
5 + 3		
print("Hello")		
age = 25		
name.upper()		
if age > 18:		
len(scores)		
for x in list:		
x > y		

Exercise 2.2: Write Conditions

Instructions: Write Python code for each scenario.

Scenario 1: Pass/Fail

- If score ≥ 60 , print "Pass"
- Otherwise, print "Fail"

Scenario 2: Age Category

- If age < 13 , print "Child"

- If age < 20, print "Teen"
- If age < 65, print "Adult"
- Otherwise, print "Senior"

Scenario 3: Discount

- If total > 100 AND is_member, apply 20% discount
- If total > 100 AND NOT is_member, apply 10% discount
- Otherwise, no discount

Exercise 2.3: Decision Tree in Python

Instructions: Draw the decision tree and write Python code for ticket pricing.

Scenario:

- Regular ticket: \$10
- Student ticket: \$8 (must be 18+ AND have student ID)
- Child ticket: \$5 (must be under 12)
- Senior ticket: \$7 (must be 65+)

Your Python code:

Exercise 2.4: Spot the Statement

Instructions: Identify statements and expressions in this Python code.

```
# Code to calculate total with discount
total = 0
for price in prices:
    total = total + price
discount = total * 0.1
final = total - discount
print(final)
```

Statements: _____

Expressions: _____

Rewrite in FP style (only final print is statement):

Exercise 2.5: Complete Python Program

Instructions: Write a Python program for the cinema ticket system.

Scenario:

- Age input from user
- Student status input (yes/no)
- Calculate ticket price based on age and status
- Print final price

Your code:

Day 2 Summary

Key Takeaways

1. **Expressions:** Produce values. Use for ALL logic.
2. **Statements:** Perform actions. Use ONLY for input/output.
3. **Conditions:** `if` , `elif` , `else` for decisions.
4. **Logical Operators:** `and` , `or` , `not` for combining conditions.
5. **FP Rule:** Push statements to the edges. Use expressions everywhere else.

Quiz Yourself

1. What is the difference between an expression and a statement in Python?
2. How do you write an `if` statement in Python?
3. What is the difference between `if` and `elif` ?
4. How do you combine multiple conditions in Python?

5. What is the FP rule for expressions vs statements?

Day 3: Python Loops, Lists & Data Structures

Learning Objectives

By the end of Day 3, you will be able to:

- Use `for` loops to iterate over lists
- Use `while` loops for conditional repetition
- Work with Python lists (append, extend, insert, remove)
- Use dictionaries for key-value data
- Combine lists and dictionaries into data structures

3.1 Python Loops

`for` Loop (FOR EACH)

Used to iterate over a sequence (list, string, tuple, dictionary).

Syntax:

```
for item in sequence:
    # do something with item
```

Examples:

Example 1: Loop over a list

```
names = ["Alice", "Bob", "Charlie"]
for name in names:
    print(f"Hello, {name}")
```

```
# Output:
# Hello, Alice
# Hello, Bob
# Hello, Charlie
```

Example 2: Loop over a string

```
word = "Python"  
for letter in word:  
    print(letter)
```

Output: P, y, t, h, o, n

Example 3: Loop with range()

```
for i in range(5):  
    print(i)
```

Output: 0, 1, 2, 3, 4

Example 4: Loop with enumerate()

```
fruits = ["apple", "banana", "cherry"]  
for index, fruit in enumerate(fruits):  
    print(f"{index}: {fruit}")
```

Output:
0: apple
1: banana
2: cherry

while Loop

Used when you need to repeat until a condition changes.

Syntax:

```
while condition:  
    # do something  
    # update condition
```

Examples:

Example 1: Countdown

```
count = 5  
while count > 0:  
    print(count)  
    count = count - 1
```

```
print("Blast off!")
```

```
# Output: 5, 4, 3, 2, 1, Blast off!
```

Example 2: Password validation

```
password = ""  
while password != "1234":  
    password = input("Enter password: ")  
print("Access granted!")
```

Example 3: Sum until condition

```
total = 0  
num = 1  
while num <= 10:  
    total = total + num  
    num = num + 1  
print(total) # 55
```

Loop Control

break : Exit the loop immediately

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)  
# Output: 0, 1, 2, 3, 4
```

continue : Skip to the next iteration

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)  
# Output: 0, 1, 3, 4
```

3.2 Python Lists

Creating Lists

```
# Empty list
empty = []

# List of numbers
numbers = [1, 2, 3, 4, 5]

# List of strings
fruits = ["apple", "banana", "cherry"]

# Mixed list
mixed = [1, "hello", True, 3.14]

# List from range
range_list = list(range(5)) # [0, 1, 2, 3, 4]
```

Accessing List Elements

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

# Indexing (starts at 0)
print(fruits[0])    # apple
print(fruits[2])    # cherry
print(fruits[-1])   # elderberry (last element)
print(fruits[-2])   # date (second last)

# Slicing [start:end:step]
print(fruits[1:3])  # ['banana', 'cherry']
print(fruits[:3])   # ['apple', 'banana', 'cherry']
print(fruits[2:])   # ['cherry', 'date', 'elderberry']
print(fruits[::-2]) # ['apple', 'cherry', 'elderberry']
```

Modifying Lists (with caution)

Python lists are mutable, but we'll practice immutability:

Immutability Approach (FP):

```
# Instead of modifying, create a new list
fruits = ["apple", "banana", "cherry"]
new_fruits = fruits + ["date"] # Create new list
# fruits remains unchanged: ['apple', 'banana', 'cherry']
```

Common List Methods (use when needed):

Method	Description	Example
<code>.append()</code>	Add to end	<code>fruits.append("date")</code>

Method	Description	Example
<code>.insert()</code>	Insert at position	<code>fruits.insert(1, "grape")</code>
<code>.extend()</code>	Add multiple	<code>fruits.extend(["grape", "orange"])</code>
<code>.remove()</code>	Remove first match	<code>fruits.remove("banana")</code>
<code>.pop()</code>	Remove and return	<code>last = fruits.pop()</code>
<code>.sort()</code>	Sort in place	<code>fruits.sort()</code>

Immutability versions:

```
# Instead of append
fruits = fruits + ["date"] # New list

# Instead of extend
fruits = fruits + ["grape", "orange"] # New list

# Instead of sort
sorted_fruits = sorted(fruits) # New list
```

List Comprehensions

A functional way to create lists:

```
# Squares of numbers 0-9
squares = [x**2 for x in range(10)]
print(squares) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Even numbers
evens = [x for x in range(10) if x % 2 == 0]
print(evens) # [0, 2, 4, 6, 8]

# Uppercase strings
names = ["alice", "bob", "charlie"]
upper_names = [name.upper() for name in names]
print(upper_names) # ['ALICE', 'BOB', 'CHARLIE']
```

3.3 Python Dictionaries

Creating Dictionaries

```
# Empty dictionary
empty = {}

# Dictionary with values
person = {
    "name": "Alice",
    "age": 25,
    "city": "London",
    "is_student": True
}

# From lists of keys and values
keys = ["name", "age", "city"]
values = ["Alice", 25, "London"]
person = dict(zip(keys, values))
```

Accessing Dictionary Values

```
person = {"name": "Alice", "age": 25, "city": "London"}

# Using square brackets
print(person["name"])    # Alice
print(person["age"])     # 25

# Using .get() (safer – returns None if missing)
print(person.get("city"))    # London
print(person.get("country")) # None

# Get all keys
print(person.keys())        # dict_keys(['name', 'age', 'city'])

# Get all values
print(person.values())      # dict_values(['Alice', 25, 'London'])

# Get all items (key, value pairs)
print(person.items())       # dict_items([('name', 'Alice'), ('age', 25), ('city', 'Lond
```

Modifying Dictionaries (with caution)

Immutable approach:

```
# Instead of modifying, create a new dictionary
person = {"name": "Alice", "age": 25}
new_person = {**person, "city": "London"} # Spread operator
# person remains unchanged
```

Mutable methods (use when needed):

```
person = {"name": "Alice", "age": 25}
person["city"] = "London" # Adds/updates
person.update({"country": "UK"})
del person["age"]
```

Dictionary Comprehensions

```
# Square numbers as dictionary
squares = {x: x**2 for x in range(5)}
print(squares) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Filtering
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares) # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

3.4 Python Data Structures

List of Dictionaries

```
students = [
    {"name": "Alice", "age": 20, "grades": [85, 90, 78]},
    {"name": "Bob", "age": 22, "grades": [55, 60, 65]},
    {"name": "Charlie", "age": 19, "grades": [92, 88, 94]}
]

# Access data
print(students[0]["name"]) # Alice
print(students[1]["grades"]) # [55, 60, 65]
print(students[2]["grades"][0]) # 92
```

Dictionary of Lists

```
class_data = {
    "class_name": "Math 101",
    "students": ["Alice", "Bob", "Charlie"],
    "grades": [85, 90, 78],
    "average": 84.33
}

# Access data
```

```
print(class_data["students"][0])    # Alice
print(class_data["grades"][1])      # 90
```

Nested Data Structures

```
company = {
    "name": "Tech Corp",
    "employees": [
        {"name": "Alice", "role": "Developer", "skills": ["Python", "SQL"]},
        {"name": "Bob", "role": "Designer", "skills": ["Figma", "HTML"]},
        {"name": "Charlie", "role": "Manager", "skills": ["Leadership", "Agile"]}
    ],
    "location": {
        "city": "London",
        "country": "UK",
        "address": "123 Main St"
    }
}

# Access nested data
print(company["employees"][0]["skills"][0])  # Python
print(company["location"]["city"])           # London
```

Day 3 Exercises

Exercise 3.1: Python For Loop

Instructions: Write a Python for loop for each scenario.

Scenario 1: Print each name in a list with "Hello, ".

```
names = ["Alice", "Bob", "Charlie"]
```

```
# Your code:
```

Scenario 2: Double each number in a list and create a new list.

```
numbers = [1, 2, 3, 4, 5]
```

```
# Your code:
```

Scenario 3: Sum all numbers in a list.

```
numbers = [10, 20, 30, 40, 50]
```

```
# Your code:
```

Exercise 3.2: Python While Loop

Instructions: Write a Python while loop for each scenario.

Scenario 1: Countdown from 5 to 1.

Scenario 2: Sum numbers from 1 to 10.

Scenario 3: Keep asking for input until user types "exit".

Exercise 3.3: Python List Operations

Instructions: Write Python code for each task. Use immutable style where possible.

Task 1: Create a list of 5 fruits.

Task 2: Get the second fruit.

Task 3: Add "orange" to the end (create new list).

Task 4: Get the first 3 fruits.

Task 5: Check if "banana" is in the list.

Task 6: Create a sorted list (new list, don't mutate).

Exercise 3.4: Python Dictionary Operations

Instructions: Create and work with Python dictionaries.

Task 1: Create a person dictionary with name, age, and city.

Task 2: Get the person's name.

Task 3: Add a new key `is_student` (immutable approach).

Task 4: Get all keys and values.

Task 5: Loop through the dictionary and print key-value pairs.

Exercise 3.5: Complete Python Data System

Instructions: Build a complete class management system in Python.

Data:

```
students = [  
    {"name": "Alice", "grades": [85, 90, 78]},  
    {"name": "Bob", "grades": [55, 60, 65]},  
    {"name": "Charlie", "grades": [92, 88, 94]}  
]
```

Tasks:

1. Calculate the average for each student
2. Determine if each student passes (average ≥ 60)
3. Count how many students pass
4. Calculate the class average

Your code:

Day 3 Summary

Key Takeaways

1. **For Loops:** `for item in sequence:` - iterate over any iterable
2. **While Loops:** `while condition:` - repeat until condition changes
3. **Lists:** Ordered collections with indexing
4. **Dictionaries:** Key-value pairs
5. **Data Structures:** Combine lists and dictionaries for complex data
6. **FP Rule:** Prefer immutable operations (create new, don't mutate)

Quiz Yourself

1. What is the difference between `for` and `while` loops?
2. How do you access the first element of a list?
3. How do you get a value from a dictionary?
4. What is a list comprehension?
5. How do you combine lists and dictionaries?

Day 4: Python Functions, Scope & Errors

Learning Objectives

By the end of Day 4, you will be able to:

- Define and use Python functions
- Use parameters and return values
- Understand variable scope in Python
- Handle errors with try/except
- Write pure Python functions

4.1 Python Functions

Defining Functions

Syntax:

```
def function_name(parameters):  
    """Docstring – describes what the function does"""  
    # Processing  
    return result
```

Simple Function Examples

Example 1: Double a number

```
def double(x):  
    """Return x multiplied by 2"""  
    return x * 2
```

```
print(double(5))    # 10  
print(double(10))   # 20
```

Example 2: Greet someone

```
def greet(name):  
    """Return a greeting"""  
    return f"Hello, {name}!"
```

```
print(greet("Alice"))    # "Hello, Alice!"
print(greet("Bob"))      # "Hello, Bob!"
```

Example 3: Check if adult

```
def is_adult(age):
    """Return True if age >= 18, False otherwise"""
    return age >= 18

print(is_adult(25))      # True
print(is_adult(16))      # False
```

Example 4: Calculate circle area

```
def circle_area(radius):
    """Calculate area of a circle"""
    return 3.14159 * radius ** 2

print(circle_area(5))    # 78.53975
print(circle_area(10))   # 314.159
```

Functions with Multiple Parameters

```
def rectangle_area(width, height):
    return width * height

print(rectangle_area(5, 3))    # 15
print(rectangle_area(10, 20)) # 200
```

Functions with Default Parameters

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alice"))          # "Hello, Alice!"
print(greet("Bob", "Hi"))      # "Hi, Bob!"
```

Functions Returning Multiple Values

```
def min_max(numbers):
    return min(numbers), max(numbers)

smallest, largest = min_max([1, 5, 3, 9, 2])
```

```
print(smallest) # 1
print(largest)  # 9
```

Pure Functions vs Impure Functions

Pure Function:

```
def add(x, y):
    return x + y

# Same inputs always produce same output
print(add(5, 3)) # 8
print(add(5, 3)) # 8 (always the same)
```

Impure Function (avoid in FP):

```
# Uses external variable
discount_rate = 0.1

def apply_discount(price):
    return price * (1 - discount_rate) # Depends on external variable
# Not pure: if discount_rate changes, output changes
```

Pure Function (FP Style):

```
def apply_discount(price, discount_rate):
    return price * (1 - discount_rate)

# All inputs are passed explicitly
print(apply_discount(100, 0.1)) # 90.0
```

Function Composition

```
def double(x):
    return x * 2

def add_ten(x):
    return x + 10

def square(x):
    return x ** 2

# Compose functions
```

```
result = square(add_ten(double(5)))  
print(result)  # (5*2+10)^2 = 20^2 = 400
```

Lambda Functions (Anonymous Functions)

```
# Lambda function - small one-line function  
double = lambda x: x * 2  
print(double(5))  # 10  
  
# Using lambda with map, filter, reduce  
numbers = [1, 2, 3, 4, 5]  
doubled = list(map(lambda x: x * 2, numbers))  
print(doubled)  # [2, 4, 6, 8, 10]  
  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print(evens)  # [2, 4]
```

4.2 Python Scope

What Is Scope in Python?

Scope determines where variables are accessible.

Two Types of Scope

1. Local Scope: Variables defined inside a function.

```
def greet():  
    message = "Hello"  # Local variable  
    print(message)  
  
greet()                # "Hello"  
print(message)         # NameError! message is not defined here
```

2. Global Scope: Variables defined outside all functions.

```
message = "Hello"      # Global variable  
  
def greet():  
    print(message)     # Can access global variable  
  
greet()                # "Hello"  
print(message)         # "Hello"
```

Python Scope Rules (LEGB)

1. Local - Function scope
2. Enclosing - Outer functions
3. Global - Module scope
4. Built-in - Python's built-in functions

```
# Example of LEGB
x = "global" # Global

def outer():
    x = "enclosing" # Enclosing

    def inner():
        x = "local" # Local
        print(x)

    inner()

outer() # "local"
```

Accessing Global Variables

```
count = 0 # Global

def increment():
    global count # Declare we're using global
    count = count + 1

increment()
print(count) # 1
```

Best Practice: Avoid Global Variables

```
# BAD: Using globals
tax_rate = 0.1 # Global

def calculate_total(price):
    return price * (1 + tax_rate)

# GOOD: Pass as parameter
def calculate_total(price, tax_rate):
    return price * (1 + tax_rate)
```

4.3 Python Errors and Exceptions

Types of Errors in Python

1. **SyntaxError**: Invalid Python syntax

```
print("Hello" # SyntaxError: missing closing )
```

2. **NameError**: Variable not defined

```
print(age) # NameError: name 'age' is not defined
```

3. **TypeError**: Wrong type used

```
5 + "3" # TypeError: unsupported operand type
```

4. **ValueError**: Invalid value

```
int("abc") # ValueError: invalid literal
```

5. **IndexError**: List index out of range

```
[1, 2, 3][5] # IndexError: list index out of range
```

6. **KeyError**: Dictionary key not found

```
{"name": "Alice"}["age"] # KeyError: 'age'
```

7. **ZeroDivisionError**: Division by zero

```
10 / 0 # ZeroDivisionError
```

Handling Errors with try/except

Basic Syntax:

```
try:  
    # Code that might raise an error
```

```
risky_code()
except ErrorType:
    # Code to handle the error
    handle_error()
```

Examples:

Example 1: Handle division by zero

```
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        return "Cannot divide by zero"

print(safe_divide(10, 2)) # 5.0
print(safe_divide(10, 0)) # "Cannot divide by zero"
```

Example 2: Handle conversion errors

```
def safe_int_convert(value):
    try:
        result = int(value)
        return result
    except ValueError:
        return 0

print(safe_int_convert("123")) # 123
print(safe_int_convert("abc")) # 0
```

Example 3: Handle multiple errors

```
def safe_access(data, key):
    try:
        return data[key]
    except KeyError:
        return "Key not found"
    except TypeError:
        return "Invalid data type"

print(safe_access({"name": "Alice"}, "age")) # "Key not found"
print(safe_access(None, "name")) # "Invalid data type"
```

Example 4: try-except-else-finally

```
def divide_with_logging(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero")
        return None
    else:
        print(f"Success: Result is {result}")
        return result
    finally:
        print("Division operation attempted")

print(divide_with_logging(10, 2))
# Success: Result is 5.0
# Division operation attempted
# 5.0
```

Raising Custom Errors

```
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    if age > 150:
        raise ValueError("Age cannot exceed 150")
    return age

try:
    validate_age(-5)
except ValueError as e:
    print(f"Error: {e}")
```

Day 4 Exercises

Exercise 4.1: Define Python Functions

Instructions: Write Python functions for each task.

Task 1: Square a number

```
def square(number):
    # Your code:
```

```
print(square(5))    # Should print 25
print(square(10))   # Should print 100
```

Task 2: Check if adult

```
def is_adult(age):
    # Your code:

print(is_adult(25))    # Should print True
print(is_adult(16))    # Should print False
```

Task 3: Find larger number

```
def max_value(a, b):
    # Your code:

print(max_value(10, 20)) # Should print 20
print(max_value(5, 3))   # Should print 5
```

Task 4: Calculate text length

```
def text_length(text):
    # Your code:

print(text_length("Hello"))    # Should print 5
print(text_length("Python"))   # Should print 6
```

Task 5: Sum all numbers in a list

```
def sum_all(numbers):
    # Your code:

print(sum_all([1, 2, 3, 4, 5])) # Should print 15
print(sum_all([10, 20, 30]))    # Should print 60
```

Exercise 4.2: Function Composition

Instructions: Compose functions together.

Given functions:

```
def capitalize(word):
    return word.capitalize()
```

```
def add_exclamation(word):  
    return word + "!"  
  
def reverse_word(word):  
    return word[::-1]
```

Task: Chain them together: capitalize → add_exclamation → reverse_word

```
result = reverse_word(add_exclamation(capitalize("hello")))  
print(result)  # Should print "!olleH"
```

Explain the order of execution:

- 1.
- 2.
- 3.

Exercise 4.3: Python Scope

Instructions: Identify what will be printed.

```
x = "global"  
  
def outer():  
    x = "outer"  
  
    def inner():  
        x = "inner"  
        print(x)  
  
    inner()  
    print(x)  
  
outer()  
print(x)
```

What is printed?

1. inner() prints: _____
2. outer() prints: _____
3. Global print: _____

Why? _____

Exercise 4.4: Error Handling

Instructions: Add error handling to these functions.

Task 1: Safe division

```
def safe_divide(a, b):  
    # Add try/except for ZeroDivisionError  
    return a / b  
  
print(safe_divide(10, 2)) # Should print 5.0  
print(safe_divide(10, 0)) # Should print "Cannot divide by zero"
```

Task 2: Safe dictionary access

```
def safe_get(data, key):  
    # Add try/except for KeyError  
    return data[key]  
  
print(safe_get({"name": "Alice"}, "name")) # Should print "Alice"  
print(safe_get({"name": "Alice"}, "age"))  # Should print "Key not found"
```

Task 3: Safe integer conversion

```
def safe_int(value):  
    # Add try/except for ValueError  
    return int(value)  
  
print(safe_int("123")) # Should print 123  
print(safe_int("abc")) # Should print 0
```

Exercise 4.5: Complete Python Function System

Instructions: Build a complete e-commerce system with functions.

Requirements:

1. `calculate_subtotal(cart)` - sum of all item prices
2. `apply_discount(subtotal, coupon)` - apply discount
3. `add_tax(total, tax_rate)` - add tax
4. `calculate_total(cart, coupon, tax_rate)` - final price

Your code:

```
def calculate_subtotal(cart):  
    # Your code:  
  
def apply_discount(subtotal, coupon):  
    # Your code:  
  
def add_tax(total, tax_rate):  
    # Your code:  
  
def calculate_total(cart, coupon, tax_rate):  
    # Your code:  
  
# Test  
cart = [{"name": "Book", "price": 20}, {"name": "Pen", "price": 5}]  
total = calculate_total(cart, "SAVE10", 0.1)  
print(total) # Should print 24.75
```

Day 4 Summary

Key Takeaways

1. **Functions:** `def function_name(parameters):` - reusable code blocks
2. **Parameters:** Inputs passed to functions
3. **Return:** Output returned by functions (`return value`)
4. **Scope:** Local (inside functions) vs Global (outside)
5. **Errors:** `SyntaxError`, `NameError`, `TypeError`, `ValueError`, etc.
6. **Error Handling:** `try / except` blocks
7. **FP Rule:** Keep functions pure (no side effects, no globals)

Quiz Yourself

1. How do you define a function in Python?
2. What is the difference between a parameter and an argument?
3. What is local scope?
4. What is a pure function?
5. How do you handle errors in Python?
6. What is the difference between `try` and `except` ?

Day 5: Python Input/Output, Algorithms & Complete Programs

Learning Objectives

By the end of Day 5, you will be able to:

- Get input from users with `input()`
- Print output with `print()`
- Work with files (read/write)
- Design algorithms in Python
- Build complete Python programs

5.1 Python Input and Output

Output: `print()`

The `print()` function outputs text to the console.

Basic usage:

```
print("Hello World")
print(42)
print(3.14)
print(True)
print([1, 2, 3])
```

Multiple arguments:

```
print("Hello", "World", 42) # Hello World 42
```

f-strings (formatting):

```
name = "Alice"
age = 25
print(f"Name: {name}, Age: {age}") # Name: Alice, Age: 25
```

Format specifiers:

```
price = 99.99
print(f"Price: ${price:.2f}") # Price: $99.99
```

Input: `input()`

The `input()` function gets user input from the keyboard.

Basic usage:

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Converting input to other types:

```
# String input
age_str = input("Enter your age: ")
age = int(age_str) # Convert to int

# Or directly
age = int(input("Enter your age: "))
```

Example: Simple calculator

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
operation = input("Enter operation (+, -, *, /): ")

if operation == "+":
    result = num1 + num2
elif operation == "-":
    result = num1 - num2
elif operation == "*":
    result = num1 * num2
elif operation == "/":
    result = num1 / num2
else:
    result = "Invalid operation"

print(f"Result: {result}")
```

File Input/Output

Writing to a file:

```
# Write text to a file
with open("output.txt", "w") as file:
    file.write("Hello World\n")
    file.write("Second line\n")

# Append to a file
with open("output.txt", "a") as file:
    file.write("Appended line\n")
```

Reading from a file:

```
# Read entire file
with open("output.txt", "r") as file:
    content = file.read()
    print(content)

# Read line by line
with open("output.txt", "r") as file:
    for line in file:
        print(line.strip()) # strip() removes newline
```

Working with CSV data:

```
# Read CSV
with open("data.csv", "r") as file:
    for line in file:
        parts = line.strip().split(",")
        print(parts)

# Write CSV
data = [{"Alice", "25"}, {"Bob", "30"}, {"Charlie", "35"}]
with open("output.csv", "w") as file:
    for row in data:
        file.write(",".join(row) + "\n")
```

5.2 Python Algorithms

Algorithm 1: Find Maximum

```
def find_max(numbers):
    """Find the largest number in a list"""
    if not numbers: # Empty list
        return None
```

```
max_value = numbers[0]
for num in numbers[1:]:
    if num > max_value:
        max_value = num
return max_value

# Test
print(find_max([3, 7, 2, 9, 1])) # 9
print(find_max([-5, -2, -8]))    # -2
```

Algorithm 2: Search for an Item

```
def find_item(items, target):
    """Search for an item in a list"""
    for item in items:
        if item == target:
            return True
    return False

# Test
print(find_item(["apple", "banana", "cherry"], "banana")) # True
print(find_item(["apple", "banana", "cherry"], "date"))    # False
```

Algorithm 3: Sort a List (Bubble Sort)

```
def bubble_sort(numbers):
    """Sort a list from smallest to largest"""
    sorted_list = numbers.copy() # Don't mutate original
    n = len(sorted_list)

    for i in range(n):
        for j in range(0, n - i - 1):
            if sorted_list[j] > sorted_list[j + 1]:
                # Swap
                sorted_list[j], sorted_list[j + 1] = sorted_list[j + 1], sorted_list[j]

    return sorted_list

# Test
print(bubble_sort([5, 2, 8, 1, 3])) # [1, 2, 3, 5, 8]
```

Algorithm 4: Calculate Average

```
def calculate_average(numbers):
    """Calculate the average of a list of numbers"""
```

```
    if not numbers:
        return 0
    total = sum(numbers)
    count = len(numbers)
    return total / count

# Test
print(calculate_average([85, 90, 78, 92])) # 86.25
```

5.3 Complete Python Program Structure

The Complete Program Template

```
"""
Program: Student Grade Calculator
Description: Calculates student grades and class averages
Author: [Your Name]
Date: [Date]
"""

# 1. Data definitions
students = [
    {"name": "Alice", "grades": [85, 90, 78]},
    {"name": "Bob", "grades": [55, 60, 65]},
    {"name": "Charlie", "grades": [92, 88, 94]}
]

# 2. Functions (Pure logic)
def calculate_average(grades):
    if not grades:
        return 0
    return sum(grades) / len(grades)

def is_passing(average):
    return average >= 60

def get_letter_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    elif average >= 60:
        return "D"
    else:
        return "F"
```

```
def process_student(student):
    avg = calculate_average(student["grades"])
    status = "Pass" if is_passing(avg) else "Fail"
    grade = get_letter_grade(avg)
    return {
        "name": student["name"],
        "average": avg,
        "status": status,
        "grade": grade
    }

def process_class(students):
    results = []
    for student in students:
        results.append(process_student(student))
    return results

def calculate_class_average(results):
    if not results:
        return 0
    total = sum(r["average"] for r in results)
    return total / len(results)

def generate_report(results, class_avg):
    report = "Class Report\n"
    report += "=" * 40 + "\n\n"

    for result in results:
        report += f"Name: {result['name']}\n"
        report += f"Average: {result['average']:.2f}\n"
        report += f"Status: {result['status']}\n"
        report += f"Grade: {result['grade']}\n"
        report += "-" * 20 + "\n"

    report += f"\nClass Average: {class_avg:.2f}\n"
    return report

# 3. Main program (Input/Output)
def main():
    # Process data
    results = process_class(students)
    class_avg = calculate_class_average(results)

    # Output
    report = generate_report(results, class_avg)
    print(report)

    # Save to file
    with open("report.txt", "w") as file:
        file.write(report)

# 4. Run the program
```

```
if __name__ == "__main__":  
    main()
```

Day 5 Exercises

Exercise 5.1: Python Input and Output

Instructions: Write a Python program that:

1. Asks the user for their name
2. Asks the user for their age
3. Prints a greeting with their name
4. Prints whether they are an adult (age ≥ 18)

Your code:

Exercise 5.2: File I/O

Instructions: Write a Python program that:

1. Creates a list of names
2. Writes the names to a file
3. Reads the names back from the file
4. Prints the names

```
names = ["Alice", "Bob", "Charlie", "David"]
```

Your code:

Exercise 5.3: Python Algorithms

Instructions: Write Python functions for each algorithm.

Algorithm 1: Find minimum

```
def find_min(numbers):  
    # Your code:
```

```
print(find_min([3, 7, 2, 9, 1])) # Should print 1
```

Algorithm 2: Count occurrences

```
def count_occurrences(items, target):  
    # Your code:  
  
print(count_occurrences([1, 2, 3, 2, 4, 2, 5], 2)) # Should print 3
```

Algorithm 3: Reverse a list

```
def reverse_list(items):  
    # Your code:  
  
print(reverse_list([1, 2, 3, 4, 5])) # Should print [5, 4, 3, 2, 1]
```

Exercise 5.4: Complete Python Program

Instructions: Build a complete Python program for the student grade calculator.

Scenario:

- Input: List of student names and grades (from user or file)
- Process: Calculate average, determine pass/fail, assign letter grades
- Output: Report with all information

Your code:

```
# Your complete program here:
```

Exercise 5.5: Final Project - Pizza Shop

Instructions: Build a complete Python program for the pizza shop.

Scenario:

You run a pizza shop. Customers can order custom pizzas.

PRICING:

- Base **pizza**: \$10

- Each **topping**: +\$2
- Large **size**: +\$5
- Family **size**: +\$10
- **Discount**: 10% **if** total > \$30
- **Tax**: 10% (applied after discount)

INPUT:

- **Size**: "small", "large", or "family"
- **Toppings**: list of toppings (e.g., ["pepperoni", "mushrooms", "olives"])
- **Coupon**: "SAVE10", "SAVE20", or none

OUTPUT:

- Order summary
- Subtotal
- Discount (**if** applied)
- Tax
- Final total

Your Python program:

```
# Your complete program here:
```

Exercise 5.6: Test Your Program

Instructions: Test your pizza shop program with these test cases.

Test Case 1: Small Pizza, 2 Toppings, No Coupon

```
size = "small"
toppings = ["pepperoni", "mushrooms"]
coupon = None
```

```
# Expected output:
# Subtotal: $14.00
# Discount: $0.00
# Tax: $1.40
# Total: $15.40
```

```
# Your test:
```

Test Case 2: Large Pizza, 3 Toppings, SAVE10

```
size = "large"
toppings = ["pepperoni", "mushrooms", "olives"]
coupon = "SAVE10"
```

```
# Expected output:  
# Subtotal: $21.00  
# Discount: -$2.10  
# Tax: $1.89  
# Total: $20.79
```

```
# Your test:
```

Test Case 3: Family Pizza, 4 Toppings, Total > \$30

```
size = "family"  
toppings = ["pepperoni", "mushrooms", "olives", "peppers"]  
coupon = None
```

```
# Expected output:  
# Subtotal: $28.00  
# Discount: -$2.80 (10% since > $30)  
# Tax: $2.52  
# Total: $27.72
```

```
# Your test:
```

Day 5 Summary

Key Takeaways

1. **Input:** `input()` for user input
2. **Output:** `print()` for console output
3. **Files:** `open()` for reading/writing files
4. **Algorithms:** Step-by-step problem solving
5. **Program Structure:** Input → Process (Functions) → Output
6. **FP Rule:** Keep I/O at edges, logic in functions

Quiz Yourself

1. How do you get user input in Python?
2. How do you print output in Python?
3. How do you read a file in Python?
4. How do you write to a file in Python?
5. What is the structure of a complete Python program?
6. What is the FP rule for input/output?

Final Assessment: Python

Comprehensive Python Program

Instructions: Write a complete Python program from scratch.

Scenario:

You are building an online ticket booking system.

Rules:

- Regular tickets: \$50
- Student discount: 20% (must have student ID)
- Senior discount: 15% (must be 65+)
- Group discount: 10% for 5+ tickets
- VIP: Free upgrade for members

Input:

- Number of tickets (int)
- Student status (bool)
- Senior status (bool)
- Member status (bool)

Output:

- Base price
- Discounts applied
- Final price
- VIP upgrade status

Your Python Program:

```
"""
```

```
Program: Ticket Booking System
```

```
Description: Calculates ticket prices with discounts
```

```
Author: [Your Name]
```

```
Date: [Date]
```

```
"""
```

```
# 1. Data definitions
```

```
# 2. Functions

# 3. Main program

# 4. Run
```

Python Quick Reference

Data Types

Type	Example	Operations
int	25	+ , - , * , / , // , % , **
float	3.14	+ , - , * , / , **
str	"Hello"	+ , * , len() , .upper() , .lower()
bool	True	and , or , not
list	[1,2,3]	